# Session 17 Loop Control Statements: Deck Feedback

Oct 6, 2026 · @Pedagogy

Four changes to the deck. Two rework the PIN story so students find the need for break from real code, one pays that story off at the break intro, and one adds a game to the pass section.

## Summary

One slide is reworked, three are new and two are folded into the new game. Every other slide stays as it is. Slide numbers follow the current deck order (Where We Left Off = 1); slide ids are in brackets.

| # | Change | Slide | Where it goes | Time |
|---|---|---|---|---|
| 1 | Rework | The PIN that keeps asking ( `pin-that-keeps-asking` ) | Slide 3, same place | 4 min |
| 2 | New | Fix it with what you know | Right after slide 3 | 3 min |
| 3 | New | The PIN, with break | Right after the Break divider (slide 7) | 1 min |
| 4 | New | Runs or crashes? (a pass game) | Right after Pass: a block that does nothing (slide 23) | 3 min |
| 5 | Remove | Pass: code with empty blocks, Pass: you cannot leave a block empty (slides 24 and 25) | Folded into rounds 1 and 2 of the game | -2 min |

Overall the changes add about 4 minutes. The last section takes them back from the recap, the quiz and the close, so the session still ends at 75 minutes.

## 1. Slide 3, The PIN that keeps asking: put the code next to the phone

Keep the phone and the lunch story. Add the Python program that runs the app on the left, so students can see why the bug happens instead of being told.

### Why change it

- Right now students see an app misbehaving but never see the code behind it, so the teacher has to explain the cause.
- With the code on screen, students can trace the bug to a line themselves. That is the habit this course is building.
- The story never comes back at break. Change 3 fixes that.

**Layout.** Code card on the left, the phone on the right (reuse the current phone frame, PIN screen and Payment Successful screen). An output pane sits under the code. Keep the three try cards, but move them under the phone.

```
correct_pin = "4821"
for attempt in range(3):
    pin = input()
    if pin == correct_pin:
        print("Payment successful")
    else:
        print("Incorrect PIN")
```

Inputs: `1111`, then `4821`, then `4821`. Tested in Python 3: it prints `Incorrect PIN`, `Payment successful`, `Payment successful`. The app takes the money twice.

### Steps (5 clicks)

| Click | Code line lit | Phone shows | Output pane | Teacher asks |
|---|---|---|---|---|
| 1 | None. Whole program shown | Pay ₹40 to Ravi Canteen | Empty | "Which line asks for the PIN? How many times can it ask?" |
| 2 | Line 3, then line 7 | Try 1 of 3, PIN 1111, red "Incorrect UPI PIN" | Incorrect PIN | Nothing. Keep the pace |
| 3 | Line 3, then line 5 | Try 2 of 3, PIN 4821, then Payment Successful ₹40 | + Payment successful | Pause 2 seconds. The room thinks it is over |
| 4 | Line 3 | Try 3 of 3, the PIN screen again | No change | "The money is paid. Why is it asking again?" Take 3 or 4 answers |

| Click | Code line lit | Phone shows | Output pane | Teacher asks |
|---|---|---|---|---|
| 5 | Line 5 | Payment Successful again. ₹80 gone from the account | + Payment successful | "Which line should have stopped this? What would you write right after line 5?" |

**Keep from the current slide**

- The junior developer line in the notes. It stops "PhonePe does not do that".
- No word break, stop early or loop control on this slide.
- Try cards: try 1 = wrong PIN (red), try 2 = paid (green), try 3 = **paid again** (red, new state).

**Remove from the current slide.** Click 5's plain-English plan ("every try, up to 3 … leave straight away") moves to the end of the new slide in change 2.

**Speaker notes, replace with:** "Five clicks. Laptops shut. Click 1: ask which line asks for the PIN. Click 4: take answers before the next click. Click 5: the app charges twice; ask which line should have stopped the loop. Do not say break."

## 2. New slide after slide 3: Fix it with what you know

Students try to fix the bug with what they already know. Their fixes work, but they are long or keep the loop running, so they feel the need for a way out before they hear the word break.

**Kicker / title:** Hook / Fix it with what you know. **Section:** Hook. **Suggested id:** `pin-fix-it`.

**Steps (4 clicks)**

| Click | On screen | Teacher does |
|---|---|---|
| 1 | The bug code from slide 3, plus a 2-minute timer. Prompt: "Fix it with what you know. Pairs, on paper." | Run the timer. Walk the room |
| 2 | Student fix A: a `while` loop with two conditions (code below) | "It works. What did we have to add?" Answer: a second condition, and `pin` set before the loop |

| Click | On screen | Teacher does |
|---|---|---|
| 3 | Student fix B: a `paid` variable (code below). The loop runs 3 times, with try 3 drawn faded | "It works. Does the loop stop after paying?" Answer: no, it still goes round, it just does nothing |
| 4 | The plan (move it here from slide 3, click 5): every try, up to 3: ask for the PIN; if it is right, pay, then **leave straight away**. The green arrow leaves from the middle of the loop | "We want one line that says leave now, right here." Then the next click goes to the title slide |

Fix A, `while` with two conditions:

```
correct_pin = "4821"
attempt = 0
pin = ""
while attempt < 3 and pin != correct_pin:
    pin = input()
    attempt = attempt + 1
    if pin == correct_pin:
        print("Payment successful")
    else:
        print("Incorrect PIN")
```

Fix B, a `paid` variable:

```
correct_pin = "4821"
paid = False
for attempt in range(3):
    if paid == False:
        pin = input()
        if pin == correct_pin:
            print("Payment successful")
            paid = True
        else:
            print("Incorrect PIN")
```

Both were tested with inputs `1111`, `4821`. Each prints `Incorrect PIN`, `Payment successful` and nothing more.

**Speaker notes:** "Two minutes, pairs, paper. Accept any fix that works. Do not say they are wrong. Say: it works, and look what it cost. If nobody gets there, show A and B as other students' fixes. Never claim it cannot be done without break: someone will prove you wrong."

This slide replaces the plain-English plan at the end of the current hook, so "Name it, then the plan" only needs the title slide and the agenda.

## 3. New slide after the Break divider (slide 7): The PIN, with break

Show break for the first time by fixing the hook's bug: the same code and the same phone, with one new line. Students see break do the job they asked for 10 minutes earlier, and only then see the general shape on slide 8.

**Kicker / title:** Loop Control Statements / The PIN, with break. **Suggested id:** `break-pin-replay`. Same layout as slide 3 (code left, phone right, try cards under the phone).

```
correct_pin = "4821"
for attempt in range(3):
    pin = input()
    if pin == correct_pin:
        print("Payment successful")
        break
    else:
        print("Incorrect PIN")
```

Inputs: `1111`, `4821`. Tested: prints `Incorrect PIN`, `Payment successful`, then the program ends.

**Steps (4 clicks)**

| Click | Code | Phone and try cards | Say |
|---|---|---|---|
| 1 | Line 6 `break` slides in, marked green "new" | Pay ₹40 screen | "One new line, right where you said." |
| 2 | Line 3, then line 8 | Try 1 wrong. Output: Incorrect PIN | Keep the pace |

| Click | Code | Phone and try cards | Say |
|---|---|---|---|
| 3 | Line 3, line 5, line 6. The green arrow goes from line 6 out of the loop, past the last line | Payment Successful ₹40, then the Done screen | "break: leave the loop now." |
| 4 | Nothing lights: there is no third try | Try 3 card becomes "never asked" (the dashed state already exists in the deck) | "Does try 3 happen?" Wait for: no |

**Also change slide 8 (`break-shape`), notes only:** "This is the shape of what we just did. Line 6 of the PIN code is the break here. Trace the green arrow with your hand."

**Speaker notes:** "Four clicks. Click 1: let them see the new line before it runs. Click 4: ask if try 3 happens. If someone says it happens but nothing is typed, go back to click 3 and trace the arrow leaving before try 3 starts."

## 4. New slide after slide 23: Runs or crashes? (a pass game)

Pass is the only section without a game, and it gets 3 minutes for 4 slides. A quick vote game covers slides 24 and 25 and adds the two traps students fall into: a comment does not count as a block, and pass, unlike break, works outside a loop.

**Why a different game from the Break or continue machine (slide 21):** the machine asks "what prints?" with 4 options. This one asks one yes/no question, so a round takes about 30 seconds and the whole room answers with thumbs. Two machines with the same rules would feel like a repeat.

**Kicker / title:** Predict, then run / Runs or crashes? **Suggested id:** `pass-runs-or-crashes`. Reuse the machine's frame: level number, code card, a big result panel and a score strip. Instead of options A to D, show two big buttons: **Runs** and **Crashes**.

**Each round, 2 clicks:** (1) code appears, 5-second countdown, room shows thumbs up for Runs or thumbs down for Crashes; (2) the program runs: green output, or the red error with the line marked, plus a one-line reason.

| Round | Code | Answer | Output or error | Reason on screen |
|---|---|---|---|---|
| 1 | The growth-stage code from slide 24: if, elif and else with empty blocks | Crashes | IndentationError: expected an indented block after 'if' statement on line 3 | Every colon needs a block under it |
| 2 | Same code with `pass` in all three blocks, input 25 | Runs | A blank line | pass fills the block and does nothing |
| 3 | `if marks >= 35:` with only `# write this later` under it | Crashes | IndentationError … on line 2 | Python skips comments, so the block is still empty |
| 4 | `for i in range(3):` with `pass` under it, then `print("Done")` | Runs | Done | The loop runs 3 times and does nothing each time |
| 5 | `pass` on line 1 with nothing around it, then `print("Hi")` | Runs | Hi | Unlike break and continue, pass works outside a loop |
| 6 | `if marks >= 35:` then `pass` at the left edge | Crashes | IndentationError … on line 2 | pass has to be indented to be in the block |

Every row was run in Python 3 and gives exactly the result shown.

Full code for rounds 1 and 2:

```
age = int(input())
output = ""
if age >= 20:
    pass
elif age > 12:
    pass
else:
    pass
print(output)
```

Round 1 is this code without the three `pass` lines. Round 3: `marks = 72`, `if marks >= 35:`, `# write this later`, `print("Checked")`.

**Do not add a round with `pass` followed by `print(i)` inside a loop.** That is the finding of worksheet task 3 ("pass changes nothing, 3 still prints"), and the game must not give it away.

**Speaker notes:** "Six rounds, about 30 seconds each. Thumbs before every click. Round 3 catches the room: most think a comment fills the block. Round 5: link back to Break outside a loop."

## 5. Every slide, in order, with new timings

The deck goes from 31 slides to 32, and the timings below still fit in 75 minutes. The recap, the quiz and the close each give back time. Checkpoints move by 1 or 2 minutes, and the hard floor moves from 66 to 68. Please update the instructor guide's minute marks to match.

| # | Slide (id) | Change | Minutes (was) |
|---|---|---|---|
| 1 | Where We Left Off ( `recap` ) | None | 0–5 (0–6), with slide 2 |
| 2 | Recap of nested loops ( `recap-bug` ) | None | |
| 3 | The PIN that keeps asking ( `pin-that-keeps-asking` ) | **Rework, change 1** | 5–9 (6–10) |
| new | Fix it with what you know ( `pin-fix-it` ) | **New, change 2** | 9–12 |
| 4 | Loop Control Statements ( `todays-session` ) | None. The plan now ends the Fix it slide, so this is only the name | 12–13 (10–12), with slide 5 |
| 5 | Agenda ( `agenda` ) | None | Checkpoint 1 at 13 (12) |
| 6 | Control Statements ( `control-statements` ) | None | 13–18 (12–16), slides 6 to 9 |
| 7 | Break divider ( `divider-break` ) | None | |

| # | Slide (id) | Change | Minutes (was) |
|---|---|---|---|
| new | The PIN, with break ( `break-pin-replay` ) | **New, change 3** | |
| 8 | Break ( `break-shape` ) | Speaker notes only, see change 3 | |
| 9 | Break, when a condition is true ( `break-in-if` ) | None | |
| 10 | Follow the break ( `follow-the-break` ) | None | 18–21 (16–19) |
| 11 | Break as the very first line ( `break-first-line` ) | None | 21–23 (19–21) |
| 12 | Break outside a loop ( `break-outside-a-loop` ) | None | 23–25 (21–23) |
| 13 | Break in Nested Loop ( `break-nested` ) | None | 25–27 (23–25). Checkpoint 2 at 27 (25) |
| 14 | Worksheet task 1 ( `task-1` ) | None | 27–35 (25–33) |
| 15 | Worksheet task 2 ( `task-2` ) | None | 35–42 (33–40). Checkpoint 3 at 42 (40) |
| 16 | Classroom Quiz ( `classroom-quiz` ) | None. Plan 5 minutes, not 6 | 42–47 (40–46) |
| 17 | A reel you swipe away ( `reel-you-swipe-away` ) | None | 47–51 (46–50) |
| 18–20 | Continue divider, shape, in an if | None | 51–54 (50–53) |
| 21 | Break or continue: what prints? ( `break-or-continue-machine` ) | None | 54–59 (53–58). Checkpoint 4 at 59 (58) |
| 22–23 | Pass divider, Pass: a block that does nothing ( `pass-shape` ) | None | 59–60 (58–61 for slides 22 to 26) |

| # | Slide (id) | Change | Minutes (was) |
|---|---|---|---|
| new | Runs or crashes? ( `pass-runs-or-crashes` ) | **New, change 4** | 60–63 |
| 24 | Pass: code with empty blocks ( `pass-growth` ) | **Remove.** Now round 1 of the game | |
| 25 | Pass: you cannot leave a block empty ( `pass-empty-blocks` ) | **Remove.** Now rounds 1 and 2 of the game | |
| 26 | Pass inside a loop ( `pass-in-loop` ) | None. Shown at the start of task 3, as now | |
| 27 | Worksheet task 3 ( `task-3` ) | None | 63–68 (61–66). Hard floor at 68 (66) |
| 28 | Exit question ( `exit-question` ) | None | 68–71 (66–69) |
| 29–31 | Takeaways, Feedback, Thank you | None | 71–75 (69–75) |

The student handout needs no changes. Both student view files (student_view and student_view_new_design) include the PIN hook and slides 24 and 25, so update them to match the deck. Task 2's note in the guide says slide 25's fix is also the fix for the task 2 error. Point that note at round 2 of the game instead.